Documentation technique

Plateforme de réservation (salles, restaurants, hôtels)

Projet de stage : cadrage, conception, développement et tests

CakePHP 5.4, PHP 8.2, MySQL, Git et GitHub

Auteur : [Ton nom]

Date : 1er novembre 2026

Dépôt : https://github.com/fleurssenahounvo-has/reservation-app

1. Introduction

Ce document retrace pas à pas tout ce qui a été fait jusqu'ici : le cadrage, l'installation de l'environnement de développement, la création du dépôt GitHub, la base de données, la conception (diagrammes et maquettes), le développement de l'application et les tests. Chaque étape contient les commandes exactes à copier et, quand c'est utile, une capture d'écran.

Il sert aussi de base pour le rapport de stage et permet de refaire l'installation sur une autre machine.

1.1 Le problème

La réservation traditionnelle est lente et manque d'information. Un client doit souvent se déplacer ou appeler pour apprendre que la salle, la table ou la chambre est déjà occupée. L'application permet à toute personne de consulter les disponibilités à distance et de réserver sans compte obligatoire. Elle aide aussi les établissements à gérer leurs réservations sans conflit.

1.2 Objectifs

- Réduire le temps nécessaire pour réserver.
- Supprimer les déplacements inutiles grâce à des disponibilités toujours exactes.
- Éviter les doubles réservations sur une même ressource et une même période.

1.3 Périmètre de la version 1

- Types d'établissements : salles (conférence, réception) et restaurants. Les hôtels et auberges viendront ensuite, avec le même moteur.
- Aucun compte n'est exigé pour réserver : le client laisse ses coordonnées directement dans la réservation.
- Trois acteurs : client, gestionnaire d'établissement, administrateur de la plateforme.

1.4 Technologies choisies

Couche	Choix et raison
Interface	HTML, CSS, JavaScript (déjà maîtrisés)
Serveur	PHP 8.2 avec CakePHP 5.4 : framework MVC maintenu, avec ORM, migrations et tests intégrés. FuelPHP a été écarté car il n'est plus guère maintenu.
Base de données	MySQL (MariaDB fournie par XAMPP) : transactions et verrous nécessaires pour éviter les conflits.
Versionnage	Git et GitHub (dépôt public, vitrine pour le stage)
Tests	PHPUnit (prévu en semaine 6)

2. Cadrage fonctionnel

2.1 Diagramme de cas d'utilisation

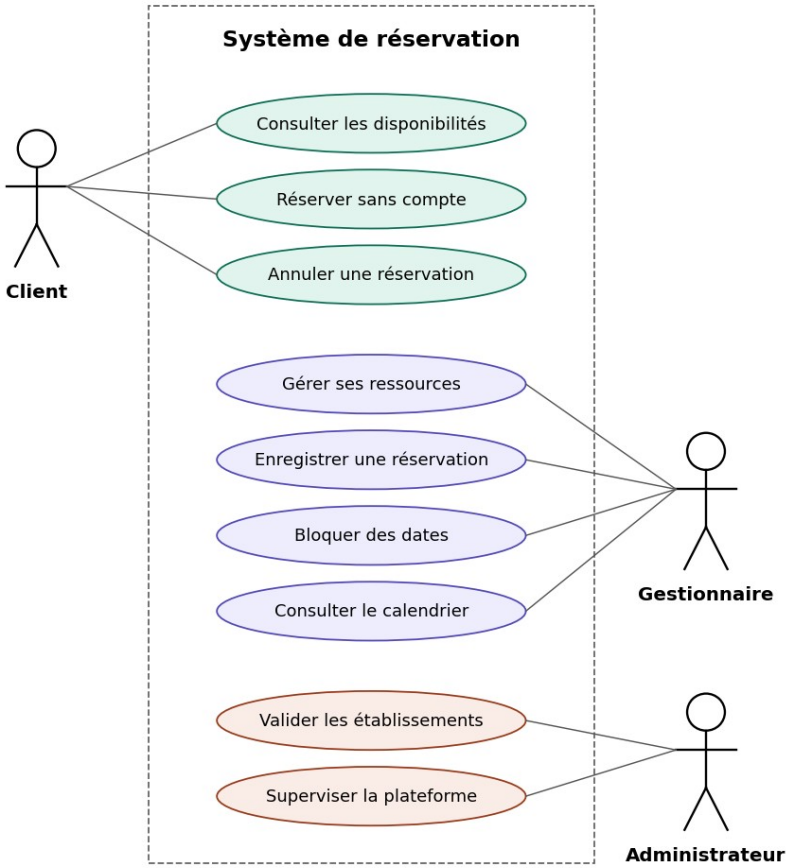


Figure 1 : cas d'utilisation du système de réservation

2.2 User stories

1. En tant que client, je veux voir les créneaux libres d'une salle ou d'une table afin d'éviter un déplacement inutile.
2. En tant que client, je veux réserver en ligne afin de gagner du temps.
3. En tant que client, je veux recevoir une confirmation afin d'être sûr que ma place est garantie.
4. En tant que gestionnaire, je veux enregistrer une réservation faite par téléphone afin que le calendrier reste à jour.
5. En tant que gestionnaire, je veux bloquer des dates (travaux, fermeture) afin qu'aucune réservation ne soit acceptée.
6. En tant que client, je veux réserver sans créer de compte afin de gagner du temps.
7. En tant que client, je veux annuler ma réservation afin de libérer le créneau si je ne peux plus venir.
8. En tant que gestionnaire, je veux ajouter mes salles, tables ou chambres avec leur capacité et leur prix afin que les clients les voient.
9. En tant que gestionnaire, je veux voir le calendrier de mes réservations afin de préparer l'accueil.
10. En tant qu'administrateur, je veux valider les établissements inscrits afin d'éviter les faux établissements.

2.3 Classement des fonctionnalités

Catégorie	Fonctionnalités
Indispensables	Liste des ressources, calendrier des disponibilités, réservation sans compte, détection des conflits, annulation, ajout d'une réservation par le gestionnaire (téléphone, comptoir), blocage de dates
Utiles	Connexion du gestionnaire, notifications par e-mail ou SMS, historique des réservations, expiration automatique des réservations en attente
Bonus	Paiement simulé, réservations récurrentes, statistiques d'occupation, comptes clients

2.4 Règles de gestion principales

- Une ressource ne peut pas avoir deux réservations actives qui se chevauchent. Deux périodes se chevauchent si `debut1 < fin2` et `debut2 < fin1`.
- Une période bloquée (fermeture, travaux) refuse toute réservation.
- Statuts d'une réservation : `en_attente`, `confirmee`, `annulee`, `expiree`.
- Sources d'une réservation : `en_ligne`, `telephone`, `comptoir`.
- En cas de demandes simultanées, une transaction avec verrouillage de la ressource garantit qu'une seule réservation passe (`SELECT ... FOR UPDATE`).

3. Installation de l'environnement

Poste utilisé : Windows, avec XAMPP (Apache, MySQL, PHP) et PowerShell.

3.1 Installer Composer

Composer est le gestionnaire de paquets de PHP. Il est téléchargé sur getcomposer.org. Pendant l'installation, l'écran « Proxy Settings » apparaît : il faut décocher la case « Use a proxy server to connect to internet », sauf si le réseau impose un proxy. Laissée cochée avec un champ vide, elle risque de provoquer des erreurs de connexion.

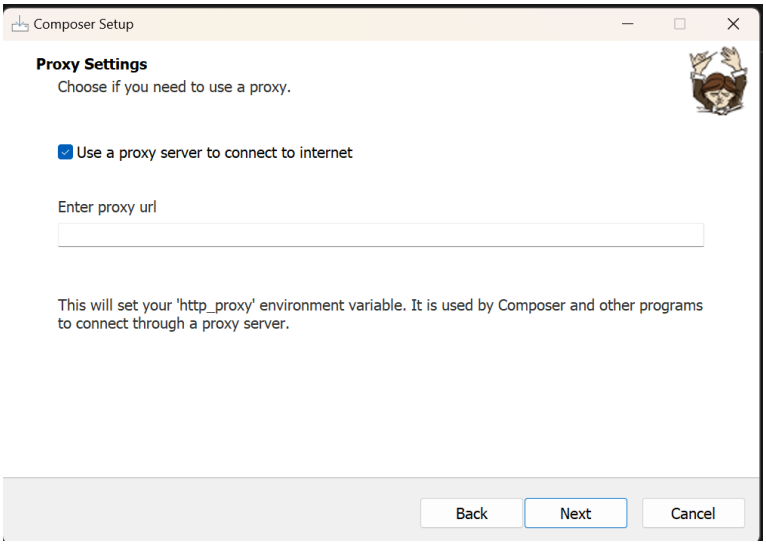


Figure 2 : écran Proxy Settings de l'installateur Composer (case à décocher)

3.2 Vérifier les versions

Après l'installation, fermer puis rouvrir le terminal, et vérifier :

Terminal

```
composer -V
php -v
```

Résultat obtenu : Composer 2.11.1 et PHP 8.2.12. CakePHP 5 demande PHP 8.1 ou plus : la version convient. Attention : `composer -v` (v minuscule) affiche l'aide complète ; la version seule s'obtient avec `-V` majuscule.

3.3 Activer les extensions PHP nécessaires

La commande `php -m` liste les extensions chargées. Dans la liste obtenue, `mbstring`, `openssl` et `pdo_mysql` étaient présentes, mais pas `intl`, que CakePHP exige. L'extension `zip` manquait aussi, ce qui a fait échouer la première installation de CakePHP avec le message « The zip extension and unzip/7z commands are both missing ».

1. Trouver le fichier php.ini réellement utilisé : `php --ini` (ligne « Loaded Configuration File »). Ici : `C:\xampp\php\php.ini`.
2. L'ouvrir : `notepad C:\xampp\php\php.ini`.
3. Chercher (Ctrl + F) les lignes `;extension=intl` et `;extension=zip`, puis supprimer le point-virgule au début pour obtenir `extension=intl` et `extension=zip`.
4. Enregistrer, fermer et rouvrir le terminal.
5. Vérifier : `php -m | findstr intl` et `php -m | findstr zip`. Chaque mot doit s'afficher.

4. Installation de CakePHP

Terminal (PowerShell)

```
cd C:\xampp\htdocs
composer create-project --prefer-dist cakephp/app reservation-app
```

À la question « Set Folder Permissions ? », répondre Y. Version installée : CakePHP 5.4.0 au téléchargement, affichée 5.4.3 dans la console.

Si une tentative échoue et laisse un dossier incomplet, le supprimer avant de relancer :

Terminal

```
Remove-Item -Recurse -Force C:\xampp\htdocs\reservation-app
```

4.1 Lancer le serveur de développement

Terminal

```
cd reservation-app
bin\cake server
```

Le terminal affiche `php server is running at http://localhost:8765/`. Ce terminal doit rester ouvert : s'il est fermé, le serveur s'arrête. Pour taper d'autres commandes, ouvrir un deuxième terminal.

Tant que le serveur n'est pas lancé (ou que le projet n'est pas installé), le navigateur affiche l'erreur ERR_CONNECTION_REFUSED :

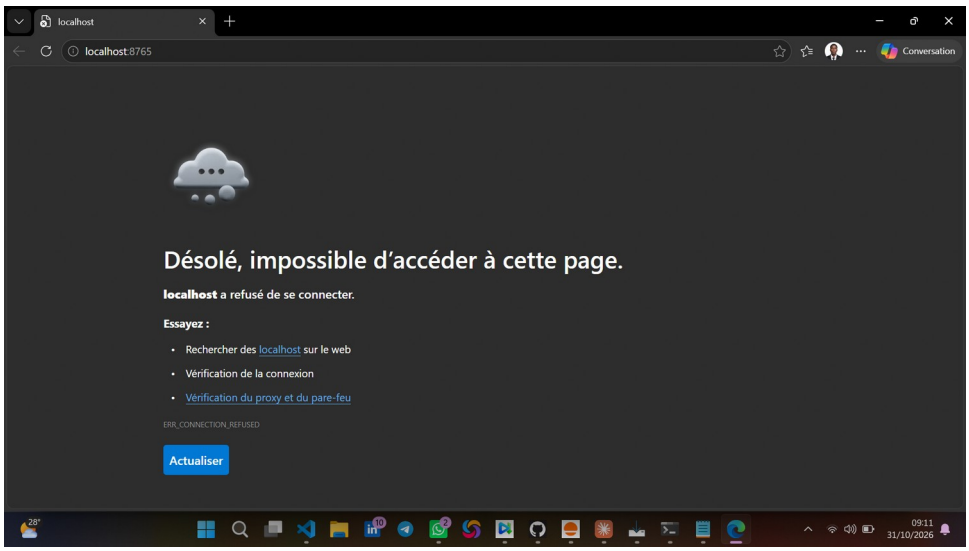


Figure 3 : erreur ERR_CONNECTION_REFUSED quand le serveur n'est pas lancé

Une fois le serveur lancé, la page d'accueil de CakePHP s'affiche à l'adresse http://localhost:8765. Toutes les vérifications de l'environnement sont au vert (PHP, mbstring, openssl, intl, dossiers tmp et logs).

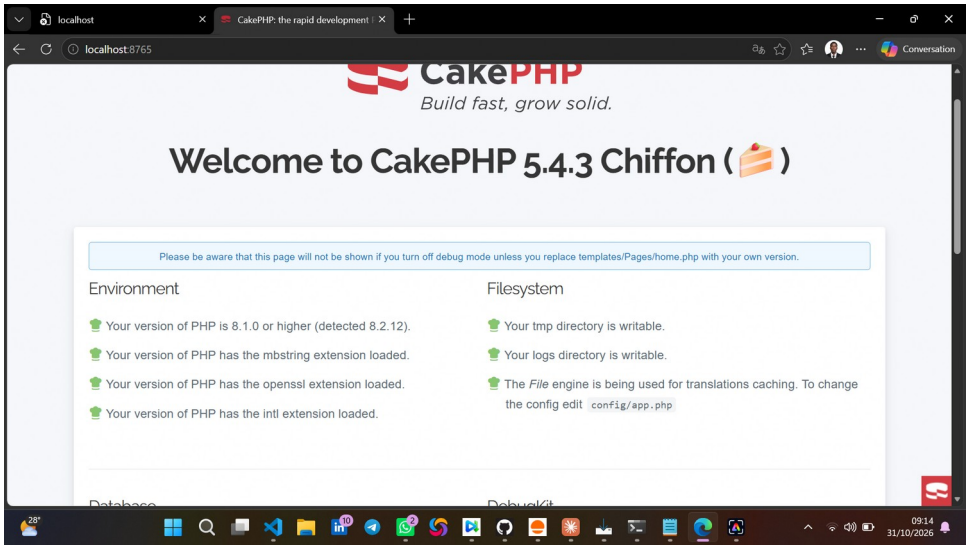


Figure 4 : page d'accueil de CakePHP 5.4.3, environnement valide

5. Git et GitHub

5.1 Création du dépôt

Dépôt public nommé `reservation-app`. Public, car il sert de vitrine pour le stage. Il ne doit jamais contenir de mots de passe ni de vraies données : le fichier `config/app_local.php`, qui porte les identifiants de base de données, est exclu par le `.gitignore` fourni avec CakePHP.

5.2 Initialisation locale

Se présenter à Git (une seule fois)

```
git config --global user.name "Ton Nom"
git config --global user.email "ton-email@exemple.com"
```

Dans C:\xampp\htdocs\reservation-app

```
git init -b main
git status
```

```
git add .
git commit -m "Initial commit : installation de CakePHP 5.4"
```

5.3 Relier au dépôt GitHub et résoudre le conflit

Terminal

```
git remote add origin https://github.com/fleurssenahounvo-has/reservation-app.git
git push -u origin main
```

Le premier `push` a été refusé (« rejected, fetch first ») : le dépôt GitHub contenait déjà un README et un .gitignore créés à la création du dépôt. Il a fallu fusionner :

Terminal

```
git pull origin main --allow-unrelated-histories --no-rebase
# conflit sur .gitignore et README.md :
git checkout --ours .gitignore      # garder la version CakePHP
git add .gitignore
git checkout --theirs README.md     # garder le README de GitHub
git add README.md
git commit -m "Fusion avec le dépôt distant"
git push -u origin main
```

Vérification sur GitHub : pas de dossier `vendor`, pas de fichier `config/app_local.php`. Seul `app_local.example.php` est présent.

Contrôle depuis le terminal

```
git ls-files config
```

6. Configuration de la base de données

6.1 Créer la base

1. Dans XAMPP Control Panel, démarrer MySQL (et Apache pour phpMyAdmin).
2. Ouvrir http://localhost/phpmyadmin.
3. Nouvelle base de données : nom `reservation_db`, interclassement `utf8mb4_unicode_ci`, puis Créer.

6.2 Configurer CakePHP

Le fichier à modifier est `config/app_local.php` (et non `app_local.example.php`, qui n'est qu'un modèle). S'il n'existe pas, le créer par copie :

Terminal

```
copy config\app_local.example.php config\app_local.php
```

Dans la section `Datasources` puis `'default'`, modifier trois lignes (autour des lignes 50 à 53) :

config/app_local.php

```
'username' => 'root',
'password' => '',
'database' => 'reservation_db',
```

La section `'test'` plus bas ne se modifie pas pour l'instant : elle servira aux tests PHPUnit de la semaine 6. L'utilisateur `root` sans mot de passe est la valeur par défaut de XAMPP et n'est acceptable qu'en local.

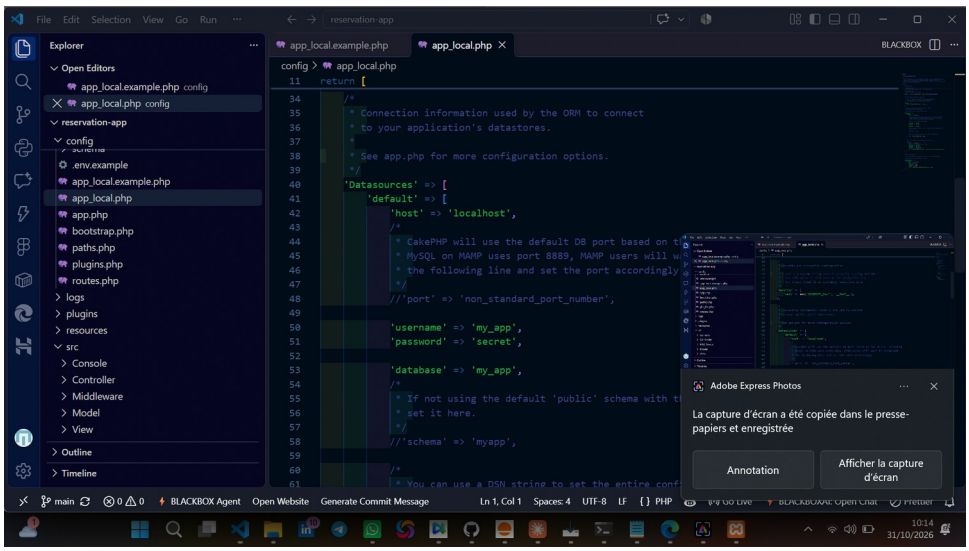


Figure 5 : fichier config/app_local.php ouvert dans VS Code (section default, lignes 50 à 53 à modifier)

6.3 Tester la connexion

En rafraîchissant http://localhost:8765, la section Database de la page d'accueil indique « CakePHP is able to connect to the database » et « DebugKit can connect to the database ».

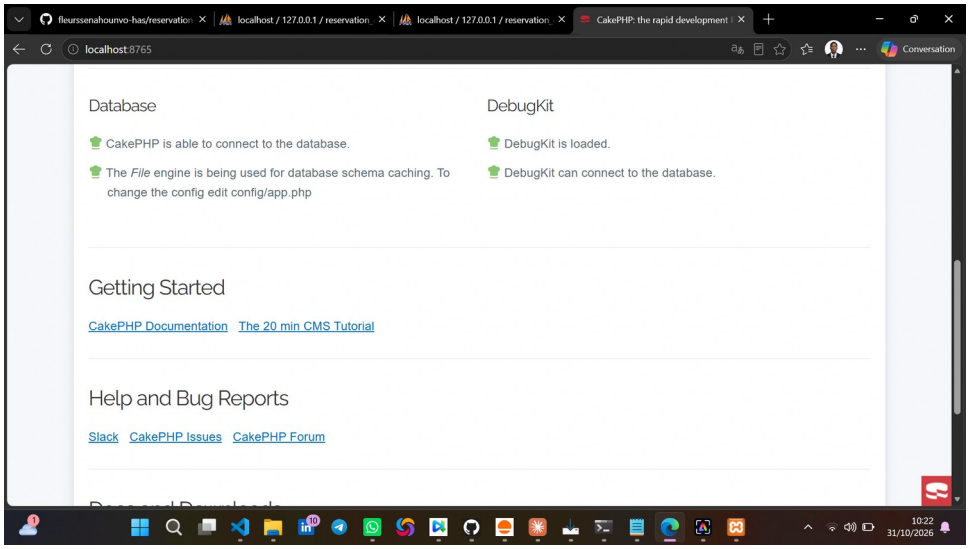


Figure 6 : connexion à la base de données réussie

7. Création des tables par migrations

Une migration est un fichier PHP qui décrit une table. Elle est versionnée dans Git, ce qui permet de recréer la base sur n'importe quelle machine. Le principe est identique pour chaque table :

Cycle de travail

```
bin\cake bake migration NomDeLaMigration champ:type ...   # génère le fichier
# modifier le fichier dans config/Migrations, puis :
bin\cake migrations migrate                               # crée la table
bin\cake migrations status                                # vérifie (up = appliquée)
git add config/Migrations
git commit -m "Migration : ..."
git push
```

Pour annuler la dernière migration (et supprimer sa table) pendant le développement : `bin\cake migrations rollback`. Dans cette installation, le registre des migrations est la table `cake_migrations`.

7.1 Table etablissements

Génération

```
bin\cake bake migration CreateEtablissements nom:string[150] type:string[30]
adresse:string[255] telephone:string[30]
```

La version générée rendait toutes les colonnes obligatoires et n'avait pas de dates. Elle a été corrigée : adresse et téléphone facultatifs, ajout de `created` et `modified`.

config/Migrations/..._CreateEtablissements.php (méthode change)

```
public function change(): void
{
    $table = $this->table('etablissements');
    $table->addColumn('nom', 'string', ['limit' => 150, 'null' => false])
          ->addColumn('type', 'string', ['limit' => 30, 'null' => false])
          ->addColumn('adresse', 'string', ['limit' => 255, 'null' => true])
          ->addColumn('telephone', 'string', ['limit' => 30, 'null' => true])
          ->addColumn('created', 'datetime', ['null' => true])
          ->addColumn('modified', 'datetime', ['null' => true])
          ->create();
}
```

Après `migrate`, phpMyAdmin affiche la table `etablissements` (vide) à côté de `cake_migrations` :

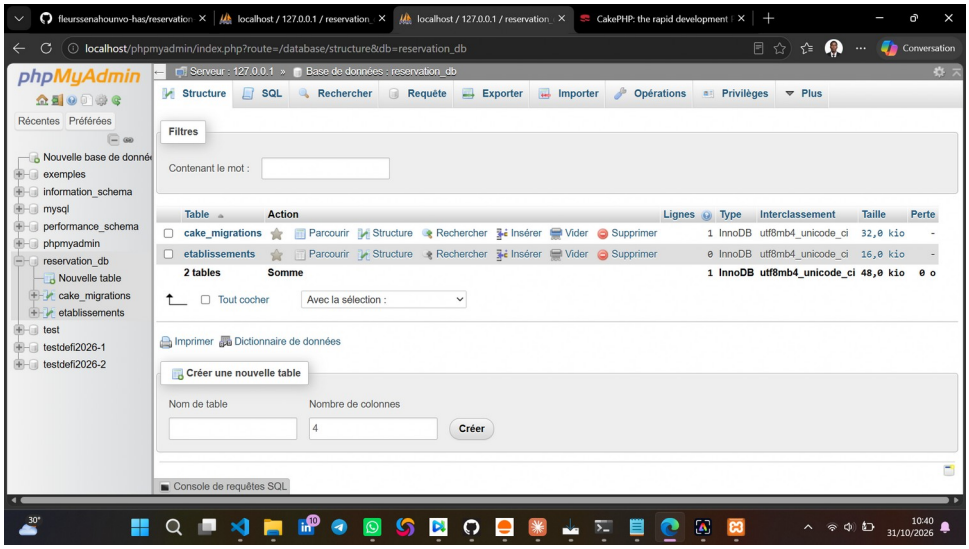


Figure 7 : base reservation_db avec les tables cake_migrations et etablissements

La requête `SELECT * FROM etablissements` confirme les 7 colonnes attendues :

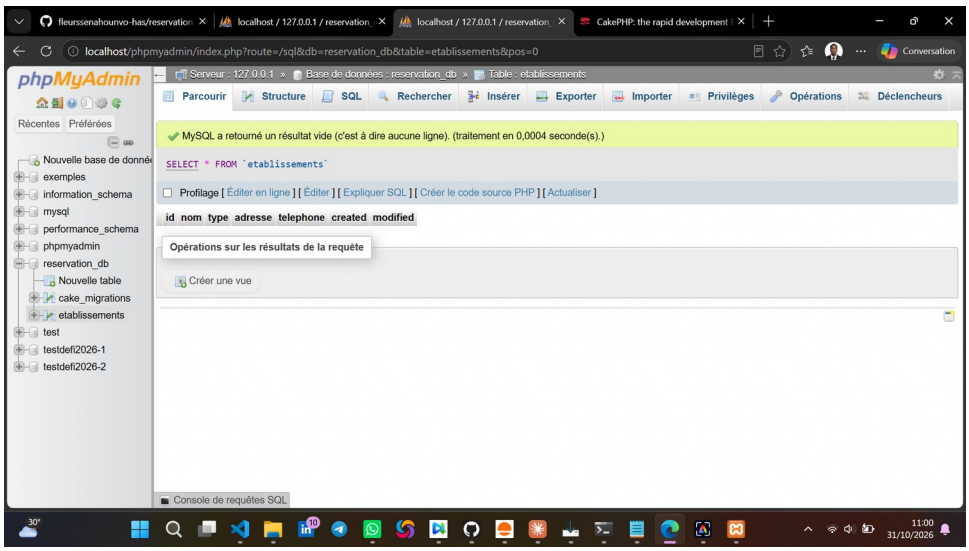


Figure 8 : colonnes de etablissements (id, nom, type, adresse, telephone, created, modified)

7.2 Table ressources (avec clé étrangère)

Génération

```
bin\cake bake migration CreateRessources etablissement_id:integer nom:string[150]
type:string[30] capacite:integer prix:decimal[10,2]
```

config/Migrations/..._CreateRessources.php (méthode change)

```
public function change(): void
{
    $table = $this->table('ressources');
    $table->addColumn('etablissement_id', 'integer', [
        'null' => false,
    ]);
    $table->addColumn('nom', 'string', ['default' => null, 'limit' => 150, 'null' =>
false]);
    $table->addColumn('type', 'string', ['default' => null, 'limit' => 30, 'null' =>
false]);
    $table->addColumn('capacite', 'integer', ['default' => null, 'limit' => 11, 'null'
=> false]);
    $table->addColumn('prix', 'decimal', [
        'default' => null, 'null' => true, 'precision' => 10, 'scale' => 2,
    ]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addForeignKey('etablissement_id', 'etablissements', 'id', [
        'delete' => 'CASCADE',
        'update' => 'NO_ACTION',
    ]);
    $table->create();
}
```

Supprimer un établissement supprime donc aussi ses ressources (`CASCADE`). Le prix est facultatif : une salle peut être sur devis. La structure obtenue dans phpMyAdmin, avec l'icône de clé étrangère sur `etablissement_id` :

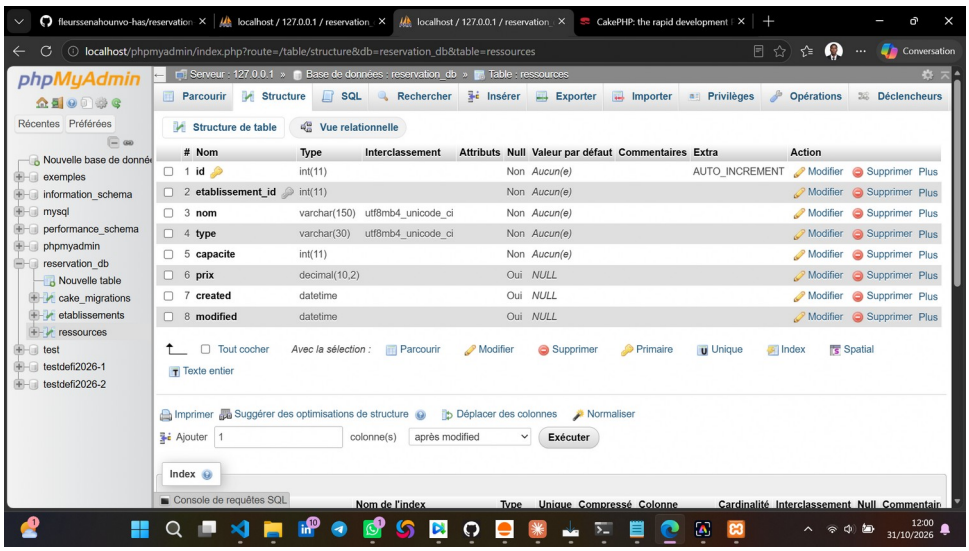


Figure 9 : structure de la table ressources

7.3 Table utilisateurs

Génération

```
bin\cake bake migration CreateUtilisateurs nom:string[100] email:string[150]
mot_de_passe:string[255] role:string[20]
```

config/Migrations/..._CreateUtilisateurs.php (méthode change)

```
public function change(): void
{
    $table = $this->table('utilisateurs');
    $table->addColumn('nom', 'string', ['default' => null, 'limit' => 100, 'null' =>
false]);
    $table->addColumn('email', 'string', ['default' => null, 'limit' => 150, 'null' =>
false]);
    $table->addColumn('mot_de_passe', 'string', ['default' => null, 'limit' => 255,
'null' => false]);
    $table->addColumn('role', 'string', ['default' => 'client', 'limit' => 20, 'null'
=> false]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addIndex(['email'], ['unique' => true]);
    $table->create();
}
```

L'index unique sur `email` interdit deux comptes avec la même adresse. La colonne `mot_de_passe` fait 255 caractères car elle stockera un mot de passe haché, jamais en clair. Rôles prévus : `client` (par défaut), `gestionnaire`, `administrateur`.

7.4 Table reservations

Décision de conception : aucun compte n'est exigé pour réserver. `utilisateur_id` est donc facultatif et les coordonnées du client sont stockées dans la réservation elle-même.

Génération

```
bin\cake bake migration CreateReservations ressource_id:integer utilisateur_id:integer
debut:datetime fin:datetime statut:string[20] source:string[20]
```

config/Migrations/..._CreateReservations.php (méthode change)

```
public function change(): void
```

```
{
    $table = $this->table('reservations');
    $table->addColumn('ressource_id', 'integer', ['null' => false]);
    $table->addColumn('utilisateur_id', 'integer', ['null' => true]);
    $table->addColumn('nom_client', 'string', ['limit' => 100, 'null' => true]);
    $table->addColumn('telephone_client', 'string', ['limit' => 30, 'null' => true]);
    $table->addColumn('email_client', 'string', ['limit' => 150, 'null' => true]);
    $table->addColumn('debut', 'datetime', ['null' => false]);
    $table->addColumn('fin', 'datetime', ['null' => false]);
    $table->addColumn('statut', 'string', ['default' => 'en_attente', 'limit' => 20,
'null' => false]);
    $table->addColumn('source', 'string', ['default' => 'en_ligne', 'limit' => 20,
'null' => false]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addIndex(['ressource_id', 'debut', 'fin']);
    $table->addForeignKey('ressource_id', 'ressources', 'id', [
        'delete' => 'CASCADE', 'update' => 'NO_ACTION',
    ]);
    $table->addForeignKey('utilisateur_id', 'utilisateurs', 'id', [
        'delete' => 'SET_NULL', 'update' => 'NO_ACTION',
    ]);
    $table->create();
}
```

L'index composé sur `(ressource_id, debut, fin)` garde rapide la recherche de chevauchements. Si un utilisateur est supprimé, ses réservations restent (`SET_NULL`).

7.5 Table blocages

Génération

```
bin\cake bake migration CreateBlocages ressource_id:integer debut:datetime fin:datetime
motif:string[255]
```

config/Migrations/..._CreateBlocages.php (méthode change)

```
public function change(): void
{
    $table = $this->table('blocages');
    $table->addColumn('ressource_id', 'integer', ['null' => false]);
    $table->addColumn('debut', 'datetime', ['null' => false]);
    $table->addColumn('fin', 'datetime', ['null' => false]);
    $table->addColumn('motif', 'string', ['limit' => 255, 'null' => true]);
    $table->addColumn('created', 'datetime', ['null' => true]);
    $table->addColumn('modified', 'datetime', ['null' => true]);
    $table->addIndex(['ressource_id', 'debut', 'fin']);
    $table->addForeignKey('ressource_id', 'ressources', 'id', [
        'delete' => 'CASCADE', 'update' => 'NO_ACTION',
    ]);
    $table->create();
}
```

7.6 État final des migrations

bin\cake migrations status

```
+--------+----------------+----------------------+
| Status | Migration ID    | Migration Name       |
+--------+----------------+----------------------+
| up     | 20261031102743  | CreateEtablissements |
```

```
| up     | 20261031105753  | CreateRessources     |
| up     | 20261031120440  | CreateUtilisateurs   |
| up     | 20261031123359  | CreateReservations   |
| up     | 20261031124210  | CreateBlocages       |
+--------+-----------------+----------------------+
```

7.7 Modèle de données

Table	Colonnes
etablissements	id, nom, type, adresse, telephone, created, modified
ressources	id, etablissement_id (clé étrangère), nom, type, capacite, prix, created, modified
utilisateurs	id, nom, email (unique), mot_de_passe (haché), role, created, modified
reservations	id, ressource_id (clé étrangère), utilisateur_id (facultatif), nom_client, telephone_client, email_client, debut, fin, statut, source, created, modified
blocages	id, ressource_id (clé étrangère), debut, fin, motif, created, modified

8. Conception (semaine 2)

La conception décrit le système avant de le coder : comment une réservation se déroule, quelles classes existent et à quoi ressemblent les écrans.

8.1 Diagramme de séquence : réserver un créneau

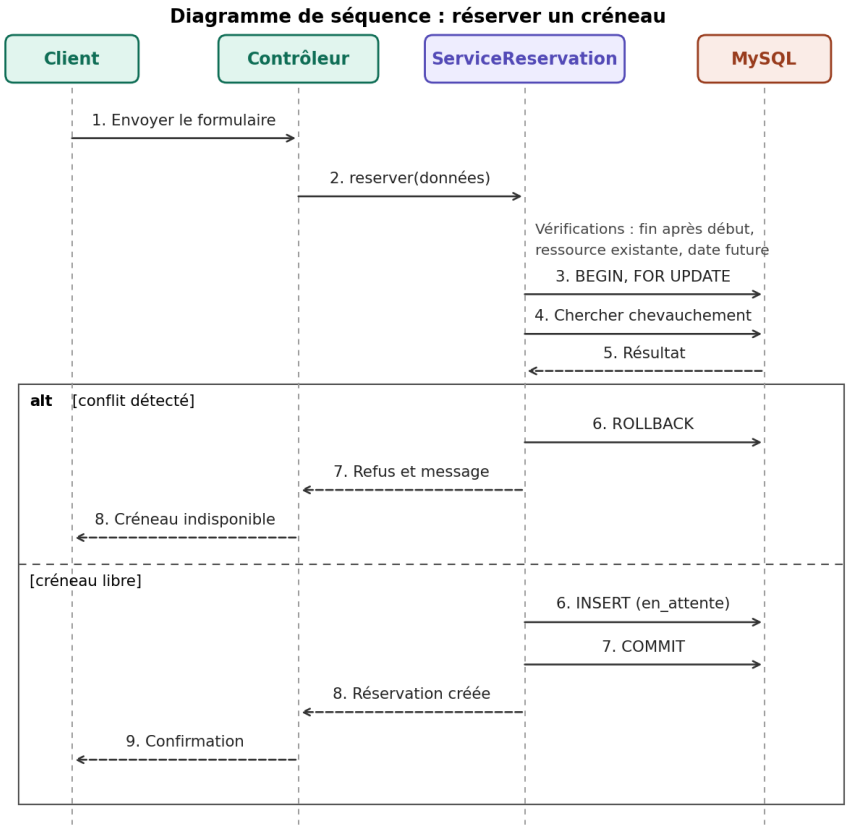


Figure 10 : séquence de la réservation d'un créneau

Le client envoie le formulaire au contrôleur, qui ne fait aucun calcul et passe les données à `ServiceReservation`. Le service vérifie les données (fin après début, ressource existante, date future), ouvre une transaction et verrouille la ressource avec `FOR UPDATE`. Pendant ce verrou, une autre demande sur la même ressource attend. Le service cherche ensuite une réservation active ou un blocage qui chevauche la période. En cas de conflit, il annule la transaction (`ROLLBACK`) et le client reçoit « Créneau indisponible ». Sinon il enregistre la réservation avec le statut `en_attente`, valide (`COMMIT`) et le client reçoit la confirmation.

Argument technique principal : sans le verrou, deux personnes pourraient vérifier « libre » au même instant, puis réserver toutes les deux. Le verrou rend la vérification et l'enregistrement indivisibles.

8.2 Diagramme de classes

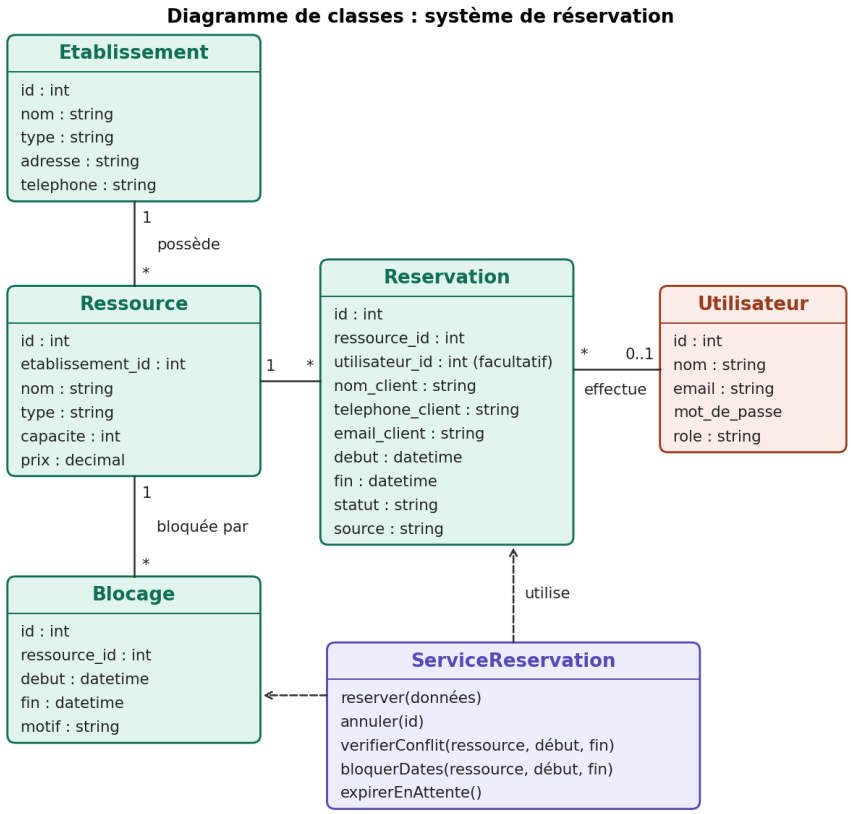


Figure 11 : diagramme de classes

Les cinq classes de données correspondent aux tables créées en semaine 1. `ServiceReservation` n'a pas de table : elle utilise les réservations et les blocages et porte toute la logique anti-conflit. Une réservation n'a pas forcément d'utilisateur (relation 0..1), puisqu'aucun compte n'est exigé pour réserver.

8.3 Maquettes des écrans

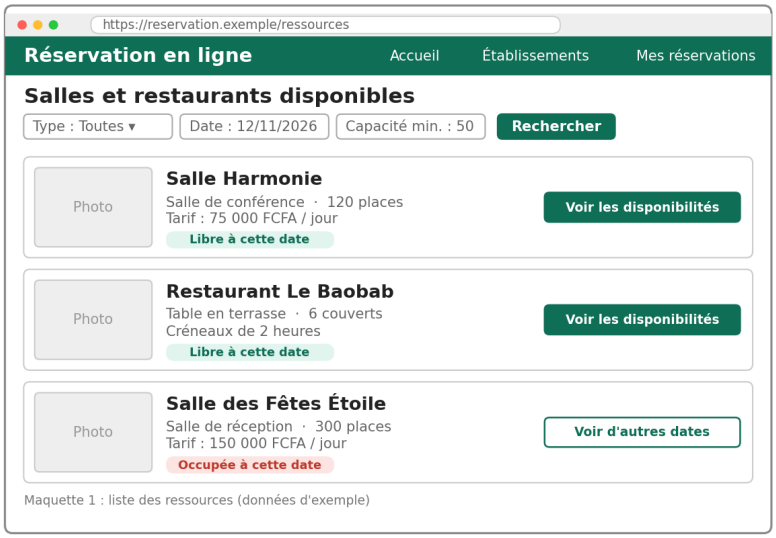


Figure 12 : maquette de la liste des ressources



Figure 13 : maquette du calendrier de disponibilité



Figure 14 : maquette du formulaire de réservation sans compte

Les noms de salles, les prix et le numéro de téléphone des maquettes sont des exemples.

9. Développement (semaines 3 à 5)

9.1 Modèles CakePHP

Pour chaque table, `bake` génère la classe Table (règles de validation et associations), la classe Entity, un jeu de données de test et un fichier de test.

Terminal

```
bin\cake bake model Etablissements
bin\cake bake model Ressources
bin\cake bake model Utilisateurs
bin\cake bake model Reservations
bin\cake bake model Blocages
```

Attention : les règles de validation générées rendaient obligatoires des champs facultatifs (adresse, téléphone, prix). Elles ont été réécrites à la main dans `validationDefault()` de chaque classe Table. Il ne faut plus relancer `bake model ... --force` sur ces classes, car cela écraserait ces corrections.

src/Model/Table/EtablissementsTable.php (extrait)

```
$validator
    ->scalar('adresse')
    ->maxLength('adresse', 255)
    ->allowEmptyString('adresse');

$validator
    ->scalar('telephone')
    ->maxLength('telephone', 30)
    ->allowEmptyString('telephone');
```

9.2 La classe ServiceReservation

Elle se trouve dans `src/Service/ServiceReservation.php` et contient toute la logique de réservation. Elle expose les méthodes suivantes :

Méthode	Rôle
`reserver()`	Valide les données, ouvre une transaction, verrouille la ressource, refuse en cas de conflit, sinon enregistre la réservation (statut en_attente par défaut, ou confirmee pour une réservation prise par téléphone ou au comptoir)
`verifierConflit()`	Vrai si le créneau chevauche une réservation active (en_attente ou confirmee) ou un blocage
`confirmer()`	Passe une réservation en attente à confirmee
`annuler()`	Passe une réservation active à annulee : le créneau redevient libre
`expirerEnAttente()`	Passe à expiree les réservations restées en attente depuis plus de 15 minutes
`bloquerDates()`	Crée un blocage (fermeture, travaux), refusé si des réservations existent déjà sur la période
`disponibilitesMois()`	Statut de chaque jour d'un mois : libre, partiel, complet, ferme ou passe

Méthode	Rôle
`creneauxJour()`	Créneaux de 2 heures d'un jour, de 08:00 à 18:00, avec leur disponibilité

Cœur de reserver() (extrait)

```
return $reservations->getConnection()->transactional(
    function () use ($reservations, $ressourceId, $debut, $fin, $donnees, $statut) {
        // Verrouille la ressource : une autre demande sur la même ressource attend
ici.
        $ressource = $this->fetchTable('Ressources')->find()
            ->where(['id' => $ressourceId])
            ->epilog('FOR UPDATE')
            ->first();
        if ($ressource === null) {
            throw new InvalidArgumentException('Ressource introuvable.');
        }

        if ($this->verifierConflit($ressourceId, $debut, $fin)) {
            throw new ConflitReservationException(
                "Ce créneau n'est plus disponible. Veuillez choisir un autre horaire."
            );
        }

        $reservation = $reservations->newEntity([
            'ressource_id' => $ressourceId,
            'nom_client' => $donnees['nom_client'] ?? null,
            'debut' => $debut,
            'fin' => $fin,
            'statut' => $statut,
            'source' => $donnees['source'] ?? 'en_ligne',
            // ... téléphone, e-mail, utilisateur_id
        ]);

        return $reservations->saveOrFail($reservation);
    }
);
```

verifierConflit() : règle de chevauchement

```
// Deux périodes se chevauchent si debut1 < fin2 et debut2 < fin1.
$reservations = $this->fetchTable('Reservations')->find()
    ->where([
        'ressource_id' => $ressourceId,
        'statut IN' => self::STATUTS_ACTIFS,   // en_attente, confirmee
        'debut <' => $fin,
        'fin >' => $debut,
    ]);
if ($reservations->count() > 0) {
    return true;
}

return $this->fetchTable('Blocages')->find()
    ->where([
        'ressource_id' => $ressourceId,
        'debut <' => $fin,
        'fin >' => $debut,
    ])
    ->count() > 0;
```

Une réservation qui commence exactement quand une autre se termine n'est pas un conflit (16:00-18:00 est accepté après 14:00-16:00). Le statut et la source d'une réservation ne viennent jamais du formulaire public : ils sont fixés par le service.

9.3 Pages publiques

La liste des ressources est la page d'accueil. Elle propose un filtre par type et par capacité minimale, et deux boutons par ressource : « Voir les disponibilités » (calendrier) et « Réserver » (formulaire). Seules les ressources d'établissements validés par l'administrateur y apparaissent.



Figure 15 : liste des ressources (page d'accueil)

Le calendrier montre chaque jour d'un mois : libre (vert), partiel (orange), complet (rouge), fermé (gris foncé) ou passé (gris clair). Un clic sur un jour libre ou partiel affiche ses créneaux de 2 heures, avec un bouton « Réserver » qui pré-remplit le formulaire. Dans la capture, le 12 novembre est partiel : 14:00-16:00 et 16:00-18:00 sont indisponibles, et le 25 et le 26 sont fermés (blocage « Travaux »).

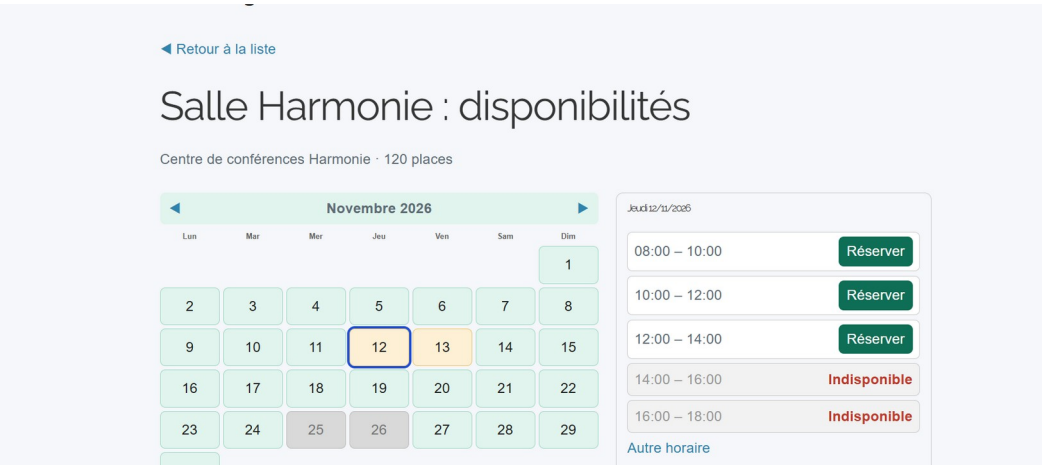


Figure 16 : calendrier de novembre 2026 et créneaux du 12 novembre

Le formulaire de réservation demande le début, la fin, le nom et le téléphone (l'e-mail est facultatif). Aucun compte n'est nécessaire. Si le créneau est déjà pris, le message de conflit s'affiche et le formulaire garde les valeurs saisies :



Figure 17 : refus d'un créneau déjà réservé (12/11/2026, 14:00-16:00)

Les erreurs de saisie sont aussi gérées, par exemple une fin avant le début :

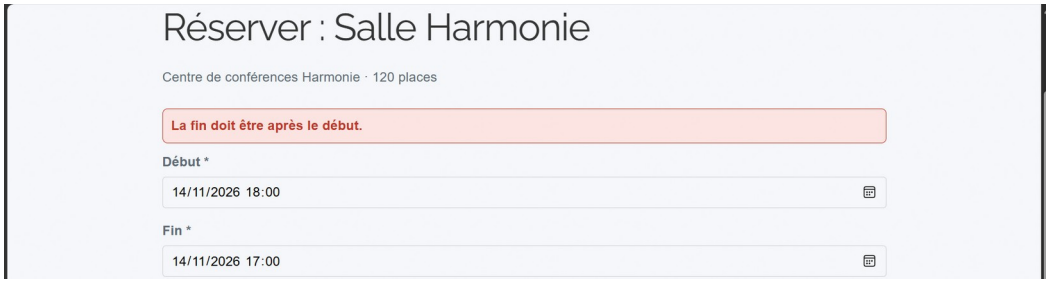


Figure 18 : refus d'une fin antérieure au début

Quand la réservation est acceptée, la page de confirmation indique son numéro et son statut « en attente de confirmation ». Elle n'affiche volontairement ni le nom, ni le téléphone, ni l'e-mail du client : sans compte, n'importe qui pourrait changer le numéro dans l'adresse et lire les réservations des autres.



Figure 19 : confirmation de la réservation n° 4

9.4 Authentification des gestionnaires

L'authentification utilise le plugin `cakephp/authentication` (version 4.2). Les mots de passe sont hachés avec `DefaultPasswordHasher` dans l'entité `Utilisateur` : ils ne sont jamais stockés en clair. Trois rôles existent : `client` (valeur par défaut, sans compte), `gestionnaire` (rattaché à un établissement par `etablissement_id`) et `administrateur`. Les pages publiques sont déclarées avec `allowUnauthenticated()` ; toutes les autres exigent une connexion.

src/Application.php : service d'authentification (plugin 4.x)

```
$champs = ['username' => 'email', 'password' => 'mot_de_passe'];

$service->loadAuthenticator('Authentication.Session');
$service->loadAuthenticator('Authentication.Form', [
    'fields' => $champs,
    'loginUrl' => $loginUrl,
    'identifier' => [
        'className' => 'Authentication.Password',
        'fields' => $champs,
        'resolver' => [
            'className' => 'Authentication.Orm',
            'userModel' => 'Utilisateurs',
```

```
        ],
    ],
]);
```

Vérification dans phpMyAdmin : le mot de passe enregistré commence par `$2y$10$`, c'est un hachage, pas le mot de passe lui-même.



Figure 20 : compte gestionnaire de test, mot de passe haché

Un compte de test se crée en ligne de commande, avec une commande écrite pour le projet :

Terminal

```
bin\cake creer_gestionnaire gestionnaire@exemple.com "Gestionnaire Test" 2
bin\cake creer_gestionnaire admin@exemple.com "Administrateur" --role administrateur
```

9.5 Espace du gestionnaire

Un gestionnaire connecté dispose de quatre pages, toutes limitées à son établissement :

- Mes réservations : liste filtrable par statut, avec les boutons Confirmer et Annuler.
- Mes ressources : ajout, modification et suppression de ses salles, tables ou chambres.
- Fermetures : blocage de périodes (travaux, fermeture), refusé si des réservations existent déjà.
- Enregistrer une réservation : pour un client qui a appelé ou qui s'est présenté sur place. La réservation est confirmée tout de suite et ne peut pas expirer.

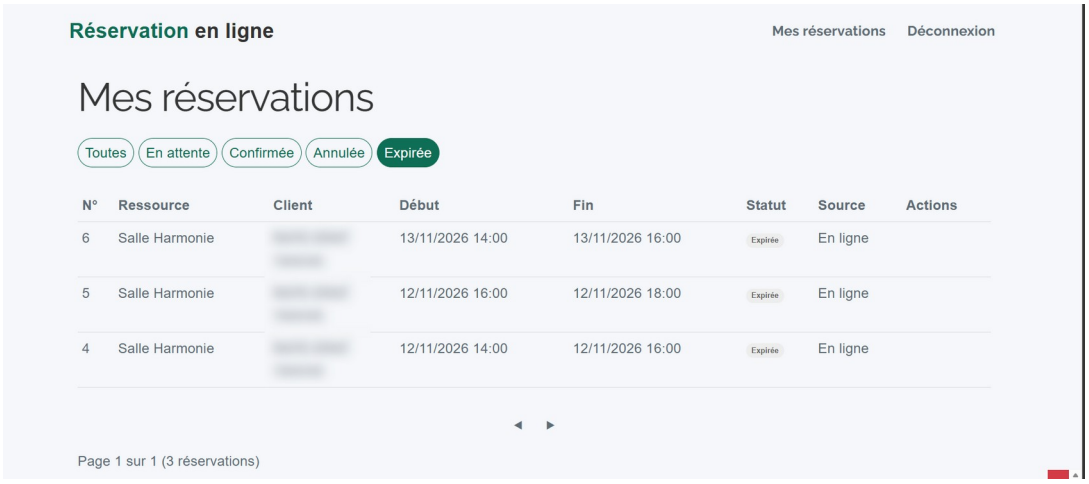


Figure 21 : « Mes réservations », filtre « Expirée » (nom et téléphone floutés)



Figure 22 : « Mes ressources » après la suppression d'une ressource de test

L'enregistrement d'une réservation par téléphone passe par la même détection de conflits que le formulaire public. Ici, une seconde tentative sur le même créneau est refusée :



Figure 23 : réservation par téléphone refusée, créneau déjà pris

Contrôle de propriété : chaque action vérifie que la ressource, la réservation ou le blocage appartient à l'établissement du gestionnaire connecté, sinon une erreur « accès refusé » (403) est renvoyée. À la création, l'établissement est imposé par le serveur et ne vient jamais du formulaire. La suppression d'une ressource est refusée s'il reste des réservations à venir, car elle effacerait ces réservations par cascade.

9.6 Inscription et validation des établissements

Un établissement s'inscrit lui-même avec son compte gestionnaire. Il reste invisible du public tant que l'administrateur ne l'a pas validé : sa liste de ressources, son calendrier et son formulaire de réservation renvoient une page « introuvable » pour le public. L'administrateur dispose d'une page pour valider ou suspendre chaque établissement.



Figure 24 : administration des établissements (validation et suspension)

9.7 Expiration des réservations en attente

Une réservation en attente bloque le créneau pendant 15 minutes. Passé ce délai, `expirerEnAttente()` la passe à `expiree` et le créneau redevient libre. Cette méthode est appelée au début de `reserver()`, de `disponibilitesMois()` et de l'ouverture de « Mes réservations ». Les réservations confirmées n'expirent jamais.

10. Tests (semaine 6)

10.1 Tests unitaires avec PHPUnit

Les tests s'exécutent sur une base séparée, `reservation_test`, pour ne jamais toucher aux vraies données. Le test refuse de s'exécuter (il est ignoré) si la base active n'est pas celle-là.

Préparation (une seule fois)

```
-- phpMyAdmin : créer la base reservation_test (utf8mb4_unicode_ci)
-- config/app_local.php, section 'test' : username root, password '', database
reservation_test,
--   'url' => env('DATABASE_TEST_URL', null)

bin\cake migrations migrate --connection test
```

Lancer les tests

```
vendor\bin\phpunit tests\TestCase\Service\ServiceReservationTest.php
```

Le fichier `tests/TestCase/Service/ServiceReservationTest.php` contient 16 tests :

Test	Résultat attendu
Créneau libre (14h-16h)	Accepté, statut en_attente
Chevauchement partiel (15h-17h)	Refusé
Chevauchement qui contient le créneau (13h-18h)	Refusé
Même créneau (14h-16h)	Refusé
Créneaux adjacents (12h-14h et 16h-18h)	Acceptés
Période bloquée	Réservation refusée
Annulation	Le créneau redevient libre
Réservation en attente depuis plus de 15 minutes	Expirée, créneau libre
Réservation confirmée depuis une heure	N'expire pas
Fin avant le début	Refusé (données invalides)
Début dans le passé	Refusé
Nom du client manquant	Refusé
Ressource inexistante	Refusé
Blocage sur une période déjà réservée	Refusé
Confirmation	Possible une seule fois, seulement depuis en_attente
Calendrier du mois	Jour libre, puis partiel, puis ferme

Premier passage : un test échoue. Une réservation créée comme « confirmée » est passée en « expirée », ce qui prouve qu'elle avait été enregistrée « en attente ».

Sortie PHPUnit, avant correction

```
........F.......                                                  16 / 16 (100%)

There was 1 failure:

1) ...ServiceReservationTest::testReservationConfirmeeNExpirePas
Failed asserting that two strings are identical.
-'confirmee'
+'expiree'

FAILURES!
Tests: 16, Assertions: 22, Failures: 1.
```

Cause : la ligne `'statut' => 'en_attente'` était restée écrite en dur dans `reserver()`, et la variable `$statut` n'était pas transmise à la fonction anonyme de la transaction (`use (...)`). Les réservations prises par téléphone ou au comptoir auraient donc expiré après 15 minutes. Les tests manuels ne le montraient pas, car il faut attendre ce délai. Après correction :

Sortie PHPUnit, après correction

```
OK (16 tests, 22 assertions)
```

10.2 Test de concurrence

Les tests unitaires lancent les opérations l'une après l'autre : ils prouvent la logique, pas le comportement quand plusieurs personnes réservent au même instant. La commande `bin\cake test_concurrence` lance plusieurs processus PHP indépendants, qui attendent le même instant (une « barrière » horaire) puis tentent de réserver le même créneau. Un seul doit réussir, et la base ne doit contenir qu'une réservation.

Terminal

```
bin\cake test_concurrence
```

Résultat : 5 manches de 6 demandes simultanées

```
OK     Manche 1 : 1 acceptée(s), 5 refusée(s), 0 erreur(s) ; réservations en base : 1
OK     Manche 2 : 1 acceptée(s), 5 refusée(s), 0 erreur(s) ; réservations en base : 1
OK     Manche 3 : 1 acceptée(s), 5 refusée(s), 0 erreur(s) ; réservations en base : 1
OK     Manche 4 : 1 acceptée(s), 5 refusée(s), 0 erreur(s) ; réservations en base : 1
OK     Manche 5 : 1 acceptée(s), 5 refusée(s), 0 erreur(s) ; réservations en base : 1
Données de test supprimées.
Résultat : jamais plus d'une réservation acceptée par créneau.
```

10.3 Limites des tests

- Le test de concurrence tourne sur une seule machine, avec une base MySQL : il ne prouve pas le comportement sous forte charge, avec plusieurs serveurs web, ni les pannes en cours de transaction.
- Il ne prouve pas à lui seul que c'est le verrou `FOR UPDATE` qui protège : pour le démontrer, il faudrait relancer le test en retirant le verrou et constater des échecs, ce qui n'est pas garanti à chaque exécution, car la collision dépend du moment exact des requêtes.
- Les pages (contrôleurs, templates) et le contrôle de propriété ont été testés à la main, pas avec des tests automatiques.

11. Problèmes rencontrés et solutions

Problème	Cause	Solution
ERR_CONNECTION_REFUSED sur localhost:8765	Le serveur n'est pas lancé, ou CakePHP n'est pas installé	Installer le projet, puis `bin\cake server` dans un terminal laissé ouvert
« The zip extension and unzip/7z commands are both missing »	Extension zip désactivée dans php.ini	Activer `extension=zip`, rouvrir le terminal, supprimer le dossier incomplet, relancer l'installation
Extension intl absente de `php -m`	Désactivée dans php.ini	Activer `extension=intl` dans `C:\xampp\php\php.ini`
`Datasources` tapé dans le terminal : commande introuvable	C'est un mot à chercher dans le fichier, pas une commande	Ouvrir `config/app_local.php` et utiliser Ctrl + F dans l'éditeur
Modification faite dans `app_local.example.php`	Mauvais fichier : c'est un modèle	Annuler (`git checkout config/app_local.example.php`) et modifier `app_local.php`
`git push` rejeté (fetch first)	Le dépôt GitHub avait déjà un commit (README, .gitignore)	`git pull origin main --allow-unrelated-histories --no-rebase`, résoudre les conflits, puis push
`git add docs` : pathspec did not match	Le dossier `docs` n'existe pas encore	`mkdir docs`, y copier le fichier, puis `git add docs`
Erreur 1005, errno 150 sur la clé étrangère de `ressources`	`etablissement_id` déclaré non signé (`'signed' => false`) alors que `etablissements.id` est `int(11)` signé	Retirer `'signed' => false` : les deux colonnes doivent avoir exactement le même type
Erreur de syntaxe dans la migration `ressources`	`$table->$table->addColumn(...)` en trop, et colonne `etablissement_id` supprimée par erreur	Recoller le fichier complet corrigé
`migrate` n'affiche pas de ligne « migrated »	Table déjà dans l'état voulu	Vérifier avec `migrations status` et phpMyAdmin
Copie depuis Téléchargements impossible (« chemin introuvable »)	Les fichiers n'étaient pas dans ce dossier	Créer les fichiers directement dans VS Code (`code chemin\fichier.php`) ou télécharger dans le dossier du projet
Fichier créé sous le nom `...Exception.ph`	Commande `code` tronquée dans le terminal	`git mv ancien.ph nouveau.php`, puis commit
Entity save failure : `adresse._required`, `telephone._required` (test, commande et PHPUnit)	Validation générée par `bake` plus stricte que la base (champs facultatifs)	Réécrire `validationDefault()` à la main avec `allowEmptyString`, ne plus relancer `bake --force`
`ConsoleArgumentParser is not available` : `bin\cake server` plante	Classe inexistante dans CakePHP 5.4 utilisée dans une commande ; CakePHP charge toutes les commandes au démarrage	Utiliser `ConsoleOptionParser`

Problème	Cause	Solution
`Call to undefined method AuthenticationService::loadIdentifier()`	Méthode supprimée dans la version 4.x du plugin d'authentification	Déclarer l'identifiant dans l'authentificateur `Form` (voir section 9.4)
`_existsIn : This value does not exist` sur `etablissement_id`	L'établissement n° 1 n'existe pas : les numéros automatiques ne sont pas réutilisés après une suppression	Consulter `SELECT id, nom FROM etablissements;` et utiliser le bon numéro
« E-mail ou mot de passe incorrect » à la première connexion	Le compte n'avait pas été créé (commande échouée)	Relancer `creer_gestionnaire`, puis vérifier le hachage (`$2y$10$`) dans phpMyAdmin
Texte vert sur fond vert dans les boutons	Le style de CakePHP écrase le style du projet pour les liens	Règles CSS plus précises (`.actions a.bouton`, états `:link` et `:visited`)
Réservations confirmées enregistrées « en attente »	`'statut' => 'en_attente'` écrit en dur et `$statut` absent du `use (...)` de la fonction anonyme	`'statut' => $statut` et `use (..., $statut)` ; défaut détecté par PHPUnit

Leçons retenues : vérifier le type exact d'une colonne avant une clé étrangère (`SHOW CREATE TABLE`) ; relire les règles de validation générées par `bake` ; ne pas se fier aux versions anciennes de la documentation d'un plugin (ici 3.x au lieu de 4.x) ; écrire des tests pour les comportements qui dépendent du temps (expiration), car on ne les voit pas en test manuel.

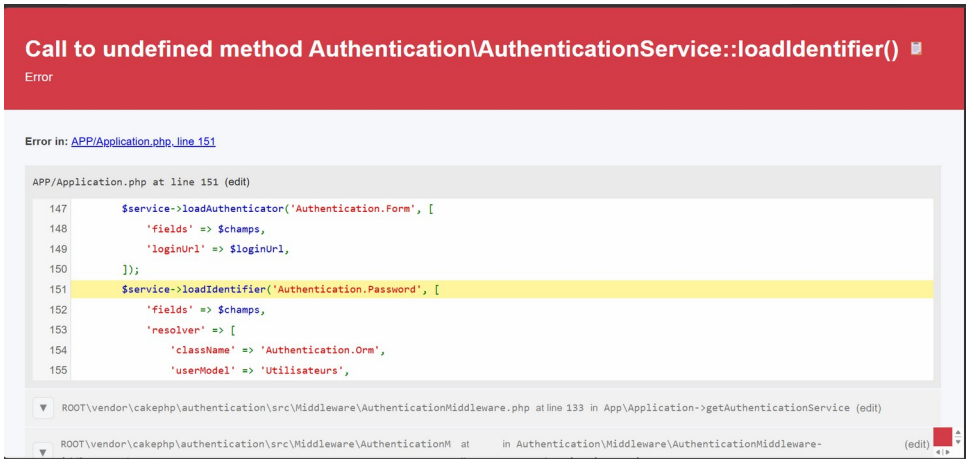


Figure 25 : erreur affichée avec le plugin d'authentification 4.x (loadIdentifier supprimée)

12. Bilan et suite

12.1 État de la version 1

Fonction	État
Liste des ressources avec filtres, calendrier, créneaux	Fait, testé à la main
Réservation sans compte	Fait, testé à la main et par PHPUnit
Détection des conflits (chevauchement, blocage)	Fait, 16 tests PHPUnit

Fonction	État
Protection contre les demandes simultanées	Fait, test de concurrence (5 manches de 6 demandes)
Expiration automatique des réservations en attente	Fait, testé par PHPUnit (déclenchée à l'ouverture des pages)
Connexion, rôles, mots de passe hachés	Fait, testé à la main
Espace gestionnaire : réservations, ressources, fermetures, réservation par téléphone	Fait, testé à la main
Contrôle de propriété (un gestionnaire ne voit que son établissement)	Fait, testé à la main
Inscription et validation des établissements	Fait, testé à la main
Déploiement en ligne	À faire (semaine 7)
Rapport et présentation	À faire (semaine 8)

12.2 Limites connues

- Les horaires (08:00 à 18:00) et les créneaux de 2 heures sont fixes dans le code ; les horaires propres à chaque établissement restent à faire.
- Les réservations en attente expirent seulement quand quelqu'un ouvre une page concernée : sur un serveur en ligne, il faudra une tâche planifiée.
- Les hôtels et auberges s'inscrivent, mais la version 1 ne gère que des créneaux horaires, pas les nuitées.
- Pas de notification par e-mail ou SMS, pas de vérification de l'adresse e-mail, pas de « mot de passe oublié ».
- Les numéros de réservation sont séquentiels, donc devinables : un code de suivi aléatoire sera nécessaire si l'on affiche des détails.
- Les fonctions de contrôle de propriété sont recopiées dans plusieurs contrôleurs ; elles pourraient devenir un composant commun.
- Une réservation prise par un gestionnaire peut avoir un début dans le passé (pour noter une réservation faite sur place).

12.3 Prochaines étapes

1. Déploiement : hébergement PHP et MySQL, mot de passe de base sécurisé, suppression des comptes de test, tâche planifiée pour l'expiration.
2. Notifications par e-mail ou SMS à la réservation et à la confirmation.
3. Gestion des nuitées pour les hôtels et auberges.
4. Horaires d'ouverture propres à chaque établissement.
5. Rapport de stage et présentation de 10 à 15 minutes avec démonstration.

Annexe : commandes essentielles

Environnement

```
php -v
composer -V
php -m | findstr intl
php --ini
```

Projet

```
cd C:\xampp\htdocs\reservation-app
bin\cake server
```

Migrations

```
bin\cake bake migration NomMigration champ:type
bin\cake migrations migrate
bin\cake migrations status
bin\cake migrations rollback
```

Tests

```
vendor\bin\phpunit tests\TestCase\Service\ServiceReservationTest.php
bin\cake test_concurrence
bin\cake migrations migrate --connection test
```

Git

```
git status
git add .
git commit -m "Message"
git push
git pull
```